

OSSP-PRACTICAL-WEEK5

Session5

Task1:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source code:

```
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>

#define BUFFER_SIZE 100
#define NUM_ITEMS 10

int main()
{
    int pipefd[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];

    struct timespec start, end;
    double elapsed_time;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    clock_gettime(CLOCK_MONOTONIC, &start);

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0)
    {

```

```

if (pid > 0)
{
    // Parent - Producer
    close(pipefd[0]);

    printf("Producer (Parent) started.\n");

    for (int i = 1; i <= NUM_ITEMS; i++)
    {
        snprintf(buffer, BUFFER_SIZE, "Data item %d", i);

        write(pipefd[1], buffer, strlen(buffer) + 1);

        printf("Producer: Sent -> %s\n", buffer);
    }

    close(pipefd[1]);

    wait(NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    elapsed_time =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("\nCommunication completed.\n");
    printf("Total communication time: %.6f seconds\n",
        elapsed_time);

    printf("Communication efficiency: %.2f items/second\n",
        NUM_ITEMS / elapsed_time);
}
else
{

```

```

        NUM_ITEMS / elapsed_time);
}
else
{
    // Child - Consumer
    close(pipefd[1]);

    printf("Consumer (Child) started.\n");

    while (read(pipefd[0], buffer, BUFFER_SIZE) > 0)
    {
        printf("Consumer: Received <- %s\n", buffer);
    }

    close(pipefd[0]);

    printf("Consumer finished.\n");
}

return 0;
}

```

Output:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ nano producer_consumer.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc producer_consumer.c
-o producer_consumer
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./producer_consumer
Producer (Parent) started.
Producer: Sent -> Data item 1
Producer: Sent -> Data item 2
Producer: Sent -> Data item 3
Consumer (Child) started.
Producer: Sent -> Data item 4
Producer: Sent -> Data item 5
Producer: Sent -> Data item 6
Consumer: Received <- Data item 1
Producer: Sent -> Data item 7
Consumer: Received <- Data item 5
Producer: Sent -> Data item 8
Consumer: Received <- Data item 8
Producer: Sent -> Data item 9
Consumer: Received <- Data item 9
Producer: Sent -> Data item 10
Consumer: Received <- Data item 10
Consumer finished.

Communication completed.
Total communication time: 0.001193 seconds
Communication efficiency: 8384.39 items/second
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

I learned how to implement communication between a parent and child process using an anonymous pipe. I understood how the process acts as the producer by writing data into the pipe, while the child process acts as the consumer by reading the data. I also learned how to measure communication time and calculate the communication rate to evaluate the efficiency of inter-process communication.

Task2:

Implement a C program to demonstrate inter-process communication using pipes, where the output of one process is connected to the input of another process, similar to the shell pipeline `ls -l | grep ".c"`.

Source code:

```
GNU nano 7.2                                pipe_pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create first child
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0)
    {
        // First child: ls -l

        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
    }
}
```

```
    close(pipefd[1]);

    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls");
    exit(EXIT_FAILURE);
}

// Create second child
pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid2 == 0)
{
    // Second child: grep ".c"

    close(pipefd[1]);

    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(EXIT_FAILURE);
}

// Parent process
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nPipeline execution completed.\n");

return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQUIUP:/mnt/c/Users/SAIRAM0806$ gcc pipe_pipeline.c -o pipe_pipeline
sairam0806@LAPTOP-M6NQUIUP:/mnt/c/Users/SAIRAM0806$ ./pipe_pipeline
lrwxrwxrwx 1 sairam0806 sairam0806      39 Aug 12  2025 Application Data -> /mnt/c/Users/SAIRAM0806/AppData/Roaming
drwxrwxrwx 1 sairam0806 sairam0806   4096 Aug 12  2025 Contacts
lrwxrwxrwx 1 sairam0806 sairam0806      67 Aug 12  2025 Cookies -> /mnt/c/Users/SAIRAM0806/AppData/Local/Microsoft/W
drwxrwxrwx 1 sairam0806 sairam0806   4096 Aug 25  2025 Documents
lrwxrwxrwx 1 sairam0806 sairam0806      37 Aug 12  2025 Local Settings -> /mnt/c/Users/SAIRAM0806/AppData/Local
drwxrwxrwx 1 sairam0806 sairam0806   4096 Aug 12  2025 Music
lrwxrwxrwx 1 sairam0806 sairam0806      33 Aug 12  2025 My Documents -> /mnt/c/Users/SAIRAM0806/Documents
-rwxrwxrwx 1 sairam0806 sairam0806  65536 Aug 12  2025 NTUSER.DAT{b98c528b-bc8a-11ef-adb3-28d043fb28de}.TM.blf
-rwxrwxrwx 1 sairam0806 sairam0806  524288 Aug 12  2025 NTUSER.DAT{b98c528b-bc8a-11ef-adb3-28d043fb28de}.TMContainer
-rwxrwxrwx 1 sairam0806 sairam0806  524288 Aug 12  2025 NTUSER.DAT{b98c528b-bc8a-11ef-adb3-28d043fb28de}.TMContainer
lrwxrwxrwx 1 sairam0806 sairam0806      75 Aug 12  2025 NetHood -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Microsoft
lrwxrwxrwx 1 sairam0806 sairam0806      75 Aug 12  2025 PrintHood -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Microso
lrwxrwxrwx 1 sairam0806 sairam0806      64 Aug 12  2025 Recent -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Microsoft/
drwxrwxrwx 1 sairam0806 sairam0806   4096 Aug 13  2025 Searches
lrwxrwxrwx 1 sairam0806 sairam0806      64 Aug 12  2025 SendTo -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Microsoft/
lrwxrwxrwx 1 sairam0806 sairam0806      68 Aug 12  2025 Start Menu -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Micros
lrwxrwxrwx 1 sairam0806 sairam0806      67 Aug 12  2025 Templates -> /mnt/c/Users/SAIRAM0806/AppData/Roaming/Microso
-rwxrwxrwx 1 sairam0806 sairam0806  16216 Sep 26 03:08 copy
-rwxrwxrwx 1 sairam0806 sairam0806   1020 Sep 26 03:02 copy.c
-rwxrwxrwx 1 sairam0806 sairam0806      85 Aug 28  2025 hello.c
-rwxrwxrwx 1 sairam0806 sairam0806  54362 Aug 23  2025 hello.c_exe
-rwxrwxrwx 1 sairam0806 sairam0806   1218 Sep 27 09:20 pipe_pipeline.c
-rwxrwxrwx 1 sairam0806 sairam0806  16224 Sep 27 08:55 practical_week4
-rwxrwxrwx 1 sairam0806 sairam0806    547 Sep 27 08:55 practical_week4.c
-rwxrwxrwx 1 sairam0806 sairam0806  16256 Sep 26 03:17 process
-rwxrwxrwx 1 sairam0806 sairam0806   1024 Sep 26 03:17 process.c
-rwxrwxrwx 1 sairam0806 sairam0806    975 Sep 25 14:26 process.c.save
-rwxrwxrwx 1 sairam0806 sairam0806  16520 Sep 27 09:08 producer_consumer
-rwxrwxrwx 1 sairam0806 sairam0806   1716 Sep 27 09:08 producer_consumer.c
-rwxrwxrwx 1 sairam0806 sairam0806     53 Sep 26 03:08 source.txt
-rwxrwxrwx 1 sairam0806 sairam0806    380 Sep 26 04:05 state.c
-rwxrwxrwx 1 sairam0806 sairam0806    380 Sep 26 03:56 state.c.save
-rwxrwxrwx 1 sairam0806 sairam0806    498 Sep 26 04:02 state.c.save.1
-rwxrwxrwx 1 sairam0806 sairam0806    561 Sep 27 08:50 zombie.c
```

Pipeline execution completed.

```
sairam0806@LAPTOP-M6NQUIUP:/mnt/c/Users/SAIRAM0806$ |
```



Observation:

I learned how to implement a shell-like pipeline using `fork()`, `pipe()`, `dup2()`, and `exec()`. I understood how the first child executes `ls -l` and sends its output through a pipe, while the second child receives the output and executes `grep ".c"`. I also learned how file descriptors can be redirected using `dup2()` and how the parent synchronizes both child processes using `waitpid()`.